Designing a Universal Information Language

Oscar Bender-Stone

March 26, 2026

DRAFT

DRAFT

Agenda

Introduction

Language Overview

Syntax

Metatheory

Conclusion

DRAFT

Background

DRAFT

[1]Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2]Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.
[3]Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

Background

- Information Management: studies how to **store** and **organize** data

DRAFT

[1]Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2]Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.
[3]Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

Background

- Information Management: studies how to **store** and **organize** data

DRAFT

[1]Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2]Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.
[3]Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

Background

- Information Management: studies how to **store** and **organize** data
  - Throughout *all* history: so much data!
  - Organize through **information**

DRAFT

[1]Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2]Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.
[3]Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

Background

- Information Management: studies how to **store** and **organize** data
  - Throughout *all* history: so much data!
  - Organize through **information**
- What *is* information?

DRAFT

[1] Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2] Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.
[3] Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

Background

- Information Management: studies how to **store** and **organize** data
  - Throughout *all* history: so much data!
  - Organize through **information**
- What *is* information?
  - Depends on who you ask!

DRAFT

[1] Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2] Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.
[3] Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

Background

- Information Management: studies how to **store** and **organize** data
  - Throughout *all* history: so much data!
  - Organize through **information**
- What *is* information?
  - Depends on who you ask!
  - **Ackoff:** Data-to-Wisdom Hierarchy[1]

DRAFT

[1] Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2] Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.
[3] Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

Background

- Information Management: studies how to **store** and **organize** data
  - Throughout *all* history: so much data!
  - Organize through **information**
- What *is* information?
  - Depends on who you ask!
  - **Ackoff:** Data-to-Wisdom Hierarchy[1]
    - Data ⇒ Information ⇒ Wisdom

DRAFT

[1] Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2] Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.
[3] Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

Background

- Information Management: studies how to **store** and **organize** data
  - Throughout *all* history: so much data!
  - Organize through **information**
- What *is* information?
  - Depends on who you ask!
  - **Ackoff:** Data-to-Wisdom Hierarchy[1]
    - Data ⇒ Information ⇒ Wisdom
  - **Buckland:** Based on usage[2]

DRAFT

[1] Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2] Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.
[3] Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

Background

- Information Management: studies how to **store** and **organize** data
  - Throughout *all* history: so much data!
  - Organize through **information**
- What *is* information?
  - Depends on who you ask!
  - **Ackoff:** Data-to-Wisdom Hierarchy[1]
    - Data ⇒ Information ⇒ Wisdom
  - **Buckland:** Based on usage[2]
    - Can be a *process*, or *knowledge*
    - Most useful as *thing*
  - **Floridi:** Information is **truthful**[3]

DRAFT

[1] Ackoff, R. L. (1989). *From Data to Wisdom*. J. Appl. Syst. Anal., 16, 3–9.
[2] Buckland, M. K. (1991). *Information as thing*. JASIS, 42(5), 351–360.
[3] Floridi, L. (2010). *Information: A Very Short Introduction*. Oxford University Press.

Problems

DRAFT

[1]Smith & Linden (2017). Two Decades of Recommender Systems at Amazon.com. IEEE Internet Comput. 21(3).
[2]Mesgari, M., et al. (2015). “The sum of all human knowledge”: A systematic review of Wikipedia. JASIST, 66(2).
[3]Reisman, M. (2017). EHRs: The challenge of making electronic data usable. P & T, 42(9), 572–575.

Problems

- How to *actually* manage information?
  - **Information base:** a database of information
- **Depth:** domain specific data
  - Example: Amazon (product suggestions)[1]

DRAFT

[1]Smith & Linden (2017). Two Decades of Recommender Systems at Amazon.com. IEEE Internet Comput. 21(3).
[2]Mesgari, M., et al. (2015). "The sum of all human knowledge": A systematic review of Wikipedia. JASIST, 66(2).
[3]Reisman, M. (2017). EHRs: The challenge of making electronic data usable. P & T, 42(9), 572–575.

Problems

- How to *actually* manage information?
  - **Information base:** a database of information
- **Depth:** domain specific data
  - Example: Amazon (product suggestions)[1]
- **Breadth:** different kinds of data

DRAFT

[1]Smith & Linden (2017). Two Decades of Recommender Systems at Amazon.com. IEEE Internet Comput. 21(3).
[2]Mesgari, M., et al. (2015). "The sum of all human knowledge": A systematic review of Wikipedia. JASIST, 66(2).
[3]Reisman, M. (2017). EHRs: The challenge of making electronic data usable. P & T, 42(9), 572–575.

Problems

- How to *actually* manage information?
  - **Information base:** a database of information
- **Depth:** domain specific data
  - Example: Amazon (product suggestions)[1]
- **Breadth:** different kinds of data
  - Example: Wikipedia[2]

DRAFT

[1]Smith & Linden (2017). Two Decades of Recommender Systems at Amazon.com. IEEE Internet Comput. 21(3).
[2]Mesgari, M., et al. (2015). "The sum of all human knowledge": A systematic review of Wikipedia. JASIST, 66(2).
[3]Reisman, M. (2017). EHRs: The challenge of making electronic data usable. P & T, 42(9), 572–575.

Problems

- How to *actually* manage information?
  - **Information base:** a database of information
- **Depth:** domain specific data
  - Example: Amazon (product suggestions)[1]
- **Breadth:** different kinds of data
  - Example: Wikipedia[2]
- **Standardization:** ambiguous interpretations
  - Related issue: being **interoperable**
  - **Interoperable:** a portable data format
  - Example: US health records[3]

DRAFT

[1] Smith & Linden (2017). Two Decades of Recommender Systems at Amazon.com. IEEE Internet Comput. 21(3).
[2] Mesgari, M., et al. (2015). "The sum of all human knowledge": A systematic review of Wikipedia. JASIST, 66(2).
[3] Reisman, M. (2017). EHRs: The challenge of making electronic data usable. P & T, 42(9), 572–575.

Existing Solutions

DRAFT

[1]Hitzler et al. (2009). *OWL 2 Web Ontology Language Primer*. W3C.
[2]Robinson, Webber & Eifrem (2015). *Graph Databases* (2nd ed.). O'Reilly.
[3]Lenat (1995). *CYC: A large-scale investment in knowledge infrastructure*. CACM 38(11).

Existing Solutions

- **Resource Description Language (RDF):**[1]
  - ▸ Used in Web Ontology Language **(OWL)**
  - \+ Built to be **interoperable**
  - — No built-in logic engine
- **Labeled Property Graphs (LPGs):**[2]
  - ▸ Stores **properties** (node/edge metadata)
  - \+ Efficient queries
  - — No logic engine. Hacky tooling/scripts
- **Cyc:**[3]
  - ▸ Largest **knowledge base** to date
  - \+ Sophisticated logic engine
  - — Hard-coded, proprietary rules

DRAFT

[1] Hitzler et al. (2009). *OWL 2 Web Ontology Language Primer*. W3C.
[2] Robinson, Webber & Eifrem (2015). *Graph Databases* (2nd ed.). O'Reilly.
[3] Lenat (1995). *CYC: A large-scale investment in knowledge infrastructure*. CACM 38(11).

Proposed Solution

DRAFT

[1] Oxford English Dictionary (2025), s.v. “welkin, n.”

Proposed Solution

- **Welkin**: an information language

DRAFT

[1] Oxford English Dictionary (2025), s.v. “welkin, n.”

Proposed Solution

- **Welkin**: an information language
  - Pronounced: “VEL-KIN”

DRAFT

[1] Oxford English Dictionary (2025), s.v. “welkin, n.”

Proposed Solution

- **Welkin**: an information language
  - Pronounced: “VEL-KIN”
  - Old German word, meaning “cloud”, “sky”[1]

DRAFT

[1] Oxford English Dictionary (2025), s.v. “welkin, n.”

Proposed Solution

- **Welkin**: an information language
  - Pronounced: “VEL-KIN”
  - Old German word, meaning “cloud”, “sky”[1]
  - Represents *expansive* nature of language

DRAFT

[1] Oxford English Dictionary (2025), s.v. “welkin, n.”

Goals

- **Universality:** be completely expressive

DRAFT

Goals

- **Universality:** be completely expressive (**done**)

DRAFT

Goals

- **Universality:** be completely expressive (**done**)
  - *Any* computer program

DRAFT

Goals

- **Universality:** be completely expressive (**done**)
  - *Any* computer program
  - *Any* “reliable” proof

DRAFT

Goals

- **Universality:** be completely expressive (**done**)
  - *Any* computer program
  - *Any* "reliable" proof
- **Scalability:** work at any size

DRAFT

Goals

- **Universality:** be completely expressive (**done**)
  - *Any* computer program
  - *Any* "reliable" proof
- **Scalability:** work at any size (**WIP**)

DRAFT

Goals

- **Universality:** be completely expressive (**done**)
  - *Any* computer program
  - *Any* "reliable" proof
- **Scalability:** work at any size (**WIP**)
  - Be efficient

DRAFT

Goals

- **Universality:** be completely expressive (**done**)
  - *Any* computer program
  - *Any* "reliable" proof
- **Scalability:** work at any size (**WIP**)
  - Be efficient
  - Support: future optimizations

DRAFT

Goals

- **Universality:** be completely expressive (**done**)
  - *Any* computer program
  - *Any* "reliable" proof
- **Scalability:** work at any size (**WIP**)
  - Be efficient
  - Support: future optimizations
  - Describe improvements *in* Welkin

DRAFT

Goals

- **Universality:** be completely expressive (**done**)
  - *Any* computer program
  - *Any* "reliable" proof
- **Scalability:** work at any size (**WIP**)
  - Be efficient
  - Support: future optimizations
  - Describe improvements *in* Welkin
- **Standardization:** be completely *unambiguous*

DRAFT

Goals

- **Universality:** be completely expressive (**done**)
  - *Any* computer program
  - *Any* "reliable" proof
- **Scalability:** work at any size (**WIP**)
  - Be efficient
  - Support: future optimizations
  - Describe improvements *in* Welkin
- **Standardization:** be completely *unambiguous* (**WIP**)

DRAFT

Goals

- **Universality:** be completely expressive (**done**)
  - *Any* computer program
  - *Any* "reliable" proof
- **Scalability:** work at any size (**WIP**)
  - Be efficient
  - Support: future optimizations
  - Describe improvements *in* Welkin
- **Standardization:** be completely *unambiguous* (**WIP**)
  - Have a formal specification

DRAFT

Agenda

Introduction

Language Overview

Syntax

Metatheory

Conclusion

DRAFT

General Requirements

DRAFT

[1]Turing, A. M. (1936). On Computable Numbers. Proc. Lond. Math. Soc., 42, 230–265.
[2]Harnad, S. (1990). *The Symbol Grounding Problem.* Physica D, 42, 335–346.

General Requirements

1. Express any computer program

DRAFT

[1]Turing, A. M. (1936). On Computable Numbers. Proc. Lond. Math. Soc., 42, 230–265.
[2]Harnad, S. (1990). *The Symbol Grounding Problem.* Physica D, 42, 335–346.

General Requirements

1. Express any computer program
   - Need effective operations

DRAFT

[1]Turing, A. M. (1936). On Computable Numbers. Proc. Lond. Math. Soc., 42, 230–265.
[2]Harnad, S. (1990). *The Symbol Grounding Problem.* Physica D, 42, 335–346.

General Requirements

1. Express any computer program
   - Need effective operations
   - Established notion: **Turing machines**[1]
2. Enable user-defined *meaning*

DRAFT

[1]Turing, A. M. (1936). On Computable Numbers. Proc. Lond. Math. Soc., 42, 230–265.
[2]Harnad, S. (1990). *The Symbol Grounding Problem.* Physica D, 42, 335–346.

General Requirements

1. Express any computer program
   - Need effective operations
   - Established notion: **Turing machines**[1]
2. Enable user-defined *meaning*
   - Avoid hollow symbols!

DRAFT

[1]Turing, A. M. (1936). On Computable Numbers. Proc. Lond. Math. Soc., 42, 230–265.
[2]Harnad, S. (1990). *The Symbol Grounding Problem.* Physica D, 42, 335–346.

General Requirements

1. Express any computer program
   - Need effective operations
   - Established notion: **Turing machines**[1]
2. Enable user-defined *meaning*
   - Avoid hollow symbols!
   - Symbol Grounding Problem[2]

DRAFT

[1] Turing, A. M. (1936). On Computable Numbers. Proc. Lond. Math. Soc., 42, 230–265.
[2] Harnad, S. (1990). *The Symbol Grounding Problem.* Physica D, 42, 335–346.

General Requirements

1. Express any computer program
   - Need effective operations
   - Established notion: **Turing machines**[1]
2. Enable user-defined *meaning*
   - Avoid hollow symbols!
   - Symbol Grounding Problem[2]

- Achieve both via **representations**

DRAFT

[1] Turing, A. M. (1936). On Computable Numbers. Proc. Lond. Math. Soc., 42, 230–265.
[2] Harnad, S. (1990). *The Symbol Grounding Problem.* Physica D, 42, 335–346.

Representations

DRAFT

Representations

- Centered around *representing* things

DRAFT

Representations

- Centered around *representing* things
- $a - c \to b$: a **represents** b **in context** c

DRAFT

Representations

- Centered around *representing* things
- $a - c \to b$: a **represents** b **in context** c
  - a: **sign**

DRAFT

Representations

- Centered around *representing* things
- $a - c \to b$: a **represents** b **in context** c
  - a: **sign**
  - b: **referent**

DRAFT

Representations

- Centered around *representing* things
- $a - c \to b$: a **represents** b **in context** c
  - a: **sign**
  - b: **referent**
  - c: **context**

DRAFT

Representations

- Centered around *representing* things
- $a - c \to b$: a **represents** b **in context** c
  - a: **sign**
  - b: **referent**
  - c: **context**
- Track *faithfulness* of representations via **truth**

DRAFT

Units

DRAFT

Units

- **Finitely defined**

DRAFT

Units

- **Finitely defined**
- Can be broken down, combined
  - Written with braces $\{a, b, \ldots, z\}$

DRAFT

Units

- **Finitely defined**
- Can be broken down, combined
  - Written with braces $\{a, b, \ldots, z\}$
- Built from **Handles**

DRAFT

Units

- **Finitely defined**
- Can be broken down, combined
  - Written with braces $\{a, b, \ldots, z\}$
- Built from **Handles**
  - Name origin: file/resource handles (programs)

DRAFT

Units

- **Finitely defined**
- Can be broken down, combined
  - Written with braces $\{a, b, \ldots, z\}$
- Built from **Handles**
  - Name origin: file/resource handles (programs)
  - Enables user-defined meaning

DRAFT

Units

- **Finitely defined**
- Can be broken down, combined
  - Written with braces $\{a, b, \ldots, z\}$
- Built from **Handles**
  - Name origin: file/resource handles (programs)
  - Enables user-defined meaning
  - User's goal: correctly specify!

DRAFT

Units

- **Finitely defined**
- Can be broken down, combined
  - Written with braces $\{a, b, \ldots, z\}$
- Built from **Handles**
  - Name origin: file/resource handles (programs)
  - Enables user-defined meaning
  - User's goal: correctly specify!
- Representations are units

DRAFT

Units

- **Finitely defined**
- Can be broken down, combined
  - Written with braces $\{a, b, \ldots, z\}$
- Built from **Handles**
  - Name origin: file/resource handles (programs)
  - Enables user-defined meaning
  - User's goal: correctly specify!
- Representations are units
- Each unit has its own **context**

DRAFT

Information

DRAFT

Information

- Representations

DRAFT

Information

- Representations $\sim$ **Truth**

DRAFT

Information

- Representations ~ **Truth**
- Information

DRAFT

Information

- Representations ~ **Truth**
- Information ~ **Proofs**

DRAFT

Information

- Representations ~ **Truth**
- Information ~ **Proofs**
  - Includes simple facts

DRAFT

Information

- Representations ~ **Truth**
- Information ~ **Proofs**
  - Includes simple facts
  - Expands to complex truth systems

DRAFT

Information

- Representations ~ **Truth**
- Information ~ **Proofs**
  - Includes simple facts
  - Expands to complex truth systems
- Major result: can express *any* "reliable" proof

DRAFT

Information

- Representations ~ **Truth**
- Information ~ **Proofs**
  - Includes simple facts
  - Expands to complex truth systems
- Major result: can express *any* "reliable" proof
  - More details later! (Metatheory)

DRAFT

Rules: Primary

ID	Name	Rule
R1	Internal Transitivity	$a - c \to b$ and $b - c \to d$ imply $a - c \to d$
R2	Pair Congruence	If $a - c \to b$, then $\{a, d\} - c \to \{b, d\}$
R3	Sign Congruence	If $a - c \to b$, then $\{a - g \to d\} - c \to \{b - g \to d\}$
R4	Context Congruence	If $a - c \to b$, then $\{d - a \to g\} - c \to \{d - b \to g\}$
R5	Referent Congruence	If $a - c \to b$, then $\{d - g \to a\} - c \to \{d - g \to b\}$
R6	Contextual Lifting	$a - c \to b$ and $d - a \to g$ imply $\{d - b \to g\} \preccurlyeq c$
R7	Refinement	$\{a - c \to b, a - c \to d\} \longrightarrow \{a - c \to b\}$

DRAFT

Rules: Empty Unit

ID	Name	Rule
R8	Empty Set	$\{g, \{\}\} \leftarrow c \rightarrow g$
R9	Sign Absorption	$\{\{\} - a \rightarrow b\} - c \rightarrow \{\}$
R10	Context Absorption	$\{a - \{\} \rightarrow b\} - c \rightarrow \{\}$
R11	Referent Absorption	$\{a - b \rightarrow \{\}\} - c \rightarrow \{\}$

DRAFT

Rules: Lattice

ID	Name	Rule
R12	Handle Equality	If $h_1 = h_2$, then $h_1 \leftarrow c \rightarrow h_2$
R13	Derivation	$\{a - g \rightarrow a\} \leftarrow c \rightarrow \{\{g, a\} \longleftrightarrow g\}$
R14	Idempotency	$\{a, a\} \leftarrow c \rightarrow a$
R15	Associativity	$\{a, \{b, c\}\} \leftarrow c \rightarrow \{\{a, b\}, c\}$
R16	Commutativity	$\{a, b\} \leftarrow c \rightarrow \{b, a\}$

DRAFT

Turing Completeness

DRAFT

Turing Completeness

- Part of Universality

DRAFT

Turing Completeness

- Part of Universality
- **Theorem:**

DRAFT

Turing Completeness

- Part of Universality
- **Theorem:** *every unit corresponds to some program, and vice versa.*
  - φ: some mapping between units and programs

DRAFT

Turing Completeness

- Part of Universality
- **Theorem:** *every unit corresponds to some program, and vice versa.*
  - φ: some mapping between units and programs
  - Interpretation:

DRAFT

Turing Completeness

- Part of Universality
- **Theorem:** *every unit corresponds to some program, and vice versa.*
  - φ: some mapping between units and programs
  - Interpretation: $a - c \to b$ iff $\varphi_c(\langle\varphi_a\rangle) = \langle\varphi_b\rangle$
  - $\langle\varphi_a\rangle$:

DRAFT

Turing Completeness

- Part of Universality
- **Theorem:** *every unit corresponds to some program, and vice versa.*
  - φ: some mapping between units and programs
  - Interpretation: $a - c \to b$ iff $\varphi_c(\langle\varphi_a\rangle) = \langle\varphi_b\rangle$
  - $\langle\varphi_a\rangle$: encodes φ_a as a string
- Later:

DRAFT

Turing Completeness

- Part of Universality
- **Theorem:** *every unit corresponds to some program, and vice versa.*
  - φ: some mapping between units and programs
  - Interpretation: $a \text{ — } c \to b$ iff $\varphi_c(\langle \varphi_a \rangle) = \langle \varphi_b \rangle$
  - $\langle \varphi_a \rangle$: encodes φ_a as a string
- Later: completeness for **proofs**!

DRAFT

Agenda

Introduction

Language Overview

Syntax

Metatheory

Conclusion

DRAFT

Overview

DRAFT

Encoding

- Characters written in `type-writer font`[1]
- Define US-ASCII: Printable (codes: 32-126) + `EOF` (code: 127)
- **Token:** collection of contiguous characters
  - Simplifies grammar

DRAFT

[1]Font used: Intel One Mono (https://github.com/intel/intel-one-mono), licensed under OFL 1.1 (https://github.com/intel/intel-one-mono/blob/main/OFL.txt).

US-ASCII

Dec.	Hex.	Glyph	Dec.	Hex.	Glyph	Dec.	Hex.	Glyph	Dec.	Hex.	Glyph	Dec.	Hex.	Glyph	Dec.	Hex.	Glyph	Dec.	Hex.	Glyph
32	20	Space	46	2E	.	60	3C	<	74	4A	J	88	58	X	102	66	f	116	74	t
33	21	!	47	2F	/	61	3D	=	75	4B	K	89	59	Y	103	67	g	117	75	u
34	22	"	48	30	0	62	3E	>	76	4C	L	90	5A	Z	104	68	h	118	76	v
35	23	#	49	31	1	63	3F	?	77	4D	M	91	5B	[	105	69	i	119	77	w
36	24	$	50	32	2	64	40	@	78	4E	N	92	5C	\	106	6A	j	120	78	x
37	25	%	51	33	3	65	41	A	79	4F	O	93	5D	]	107	6B	k	121	79	y
38	26	&	52	34	4	66	42	B	80	50	P	94	5E	^	108	6C	l	122	7A	z
39	27	'	53	35	5	67	43	C	81	51	Q	95	5F	_	109	6D	m	123	7B	{
40	28	(	54	36	6	68	44	D	82	52	R	96	60	`	110	6E	n	124	7C	|
41	29	)	55	37	7	69	45	E	83	53	S	97	61	a	111	6F	o	125	7D	}
42	2A	*	56	38	8	70	46	F	84	54	T	98	62	b	112	70	p	126	7E	~
43	2B	+	57	39	9	71	47	G	85	55	U	99	63	c	113	71	q			
44	2C	,	58	3A	:	72	48	H	86	56	V	100	64	d	114	72	r			
45	2D	-	59	3B	;	73	49	I	87	57	W	101	65	e	115	73	s			

Table 1: Printable US-ASCII codes and glyphs.

DRAFT

Character Classes

Set Name	Characters
PRINTABLE	Listed in Table 1
WHITESPACE	\t | \r | Space
DELIMITER	{ | } | ' | " | . | ,
RESERVED	DELIMITER | * | @
SQ_CHAR	Any PRINTABLE except ' and \
DQ_CHAR	Any PRINTABLE except " and \
ID_CHAR	Any PRINTABLE not in RESERVED or WHITESPACE

Table 2: Key character classes.

DRAFT

Building Blocks

- **Invertible Syntax Descriptions:** pairs of functions:[1]

DRAFT

[1] Rendel & Ostermann (2010). Invertible syntax descriptions. Haskell '10.
[2] For computer scientists: this is an AST.

Building Blocks

- **Invertible Syntax Descriptions:** pairs of functions:[1]
  - **Parser**: takes (*consumes*) tokens, converts into a data structure[2]

DRAFT

[1] Rendel & Ostermann (2010). Invertible syntax descriptions. Haskell '10.
[2] For computer scientists: this is an AST.

Building Blocks

- **Invertible Syntax Descriptions:** pairs of functions:[1]
  - **Parser**: takes (*consumes*) tokens, converts into a data structure[2]
  - **Printer**: converts a data structure into a string
  - Identity: when parser succeeds, is "reversible" with printer.
- Need **maximal munch:**

DRAFT

[1] Rendel & Ostermann (2010). Invertible syntax descriptions. Haskell '10.
[2] For computer scientists: this is an AST.

Building Blocks

- **Invertible Syntax Descriptions:** pairs of functions:[1]
  - **Parser**: takes (*consumes*) tokens, converts into a data structure[2]
  - **Printer**: converts a data structure into a string
  - Identity: when parser succeeds, is "reversible" with printer.
- Need **maximal munch:** consume tokens, not characters

DRAFT

[1] Rendel & Ostermann (2010). Invertible syntax descriptions. Haskell '10.
[2] For computer scientists: this is an AST.

Building Blocks

- **Invertible Syntax Descriptions:** pairs of functions:[1]
  - **Parser**: takes (*consumes*) tokens, converts into a data structure[2]
  - **Printer**: converts a data structure into a string
  - Identity: when parser succeeds, is "reversible" with printer.
- Need **maximal munch:** consume tokens, not characters
  - Simplifies grammar

DRAFT

[1] Rendel & Ostermann (2010). Invertible syntax descriptions. Haskell '10.
[2] For computer scientists: this is an AST.

Building Blocks (cont')

DRAFT

Building Blocks (cont')

- Let `A`, `B`: general rules

DRAFT

Building Blocks (cont')

- Let `A, B`: general rules
- `A - seq -> B`:

DRAFT

Building Blocks (cont')

- Let `A, B`: general rules
- `A - seq -> B`:
  - **Parser:** apply `A`, then `B`.

DRAFT

Building Blocks (cont')

- Let `A`, `B`: general rules
- `A - seq -> B`:
  - **Parser:** apply `A`, then `B`.
  - **Printer:** prints `AB`.

DRAFT

Building Blocks (cont')

- Let `A, B`: general rules
- `A - seq -> B`:
  - **Parser:** apply `A`, then `B`.
  - **Printer:** prints `AB`.
- `A - seq_many_till -> B`:

DRAFT

Building Blocks (cont')

- Let `A, B`: general rules
- `A - seq -> B`:
  - **Parser:** apply `A`, then `B`.
  - **Printer:** prints `AB`.
- `A - seq_many_till -> B`:
  - **Parser:** Apply `A` zero more times, *until* the `B` accepts input

DRAFT

Building Blocks (cont')

- Let `A, B`: general rules
- `A - seq -> B`:
  - **Parser:** apply `A`, then `B`.
  - **Printer:** prints `AB`.
- `A - seq_many_till -> B`:
  - **Parser:** Apply `A` zero more times, *until* the `B` accepts input
  - **Printer:** prints zero or more `A`, then `B`

DRAFT

Building Blocks (cont')

- Let `A, B`: general rules
- `A - seq -> B`:
  - **Parser:** apply `A`, then `B`.
  - **Printer:** prints `AB`.
- `A - seq_many_till -> B`:
  - **Parser:** Apply `A` zero more times, *until* the `B` accepts input
  - **Printer:** prints zero or more `A`, then `B`
- `A - lexeme -> B`:

DRAFT

Building Blocks (cont')

- Let `A, B`: general rules
- `A - seq -> B`:
  - **Parser:** apply `A`, then `B`.
  - **Printer:** prints `AB`.
- `A - seq_many_till -> B`:
  - **Parser:** Apply `A` zero more times, *until* the `B` accepts input
  - **Printer:** prints zero or more `A`, then `B`
- `A - lexeme -> B`:
  - Same as `seq`, but whitespace allowed

DRAFT

Building Blocks (cont')

- Let `A, B`: general rules
- `A - seq -> B`:
  - **Parser:** apply `A`, then `B`.
  - **Printer:** prints `AB`.
- `A - seq_many_till -> B`:
  - **Parser:** Apply `A` zero more times, *until* the `B` accepts input
  - **Printer:** prints zero or more `A`, then `B`
- `A - lexeme -> B`:
  - Same as `seq`, but whitespace allowed
- `A - lex_many_till -> B`:

DRAFT

Building Blocks (cont')

- Let `A, B`: general rules
- `A - seq -> B`:
  - **Parser:** apply `A`, then `B`.
  - **Printer:** prints `AB`.
- `A - seq_many_till -> B`:
  - **Parser:** Apply `A` zero more times, *until* the `B` accepts input
  - **Printer:** prints zero or more `A`, then `B`
- `A - lexeme -> B`:
  - Same as `seq`, but whitespace allowed
- `A - lex_many_till -> B`:
  - Same as `seq_many_till`, but whitespace allowed

DRAFT

Building Blocks (cont')

- Let `A, B`: general rules
- `A - seq -> B`:
  - **Parser:** apply `A`, then `B`.
  - **Printer:** prints `AB`.
- `A - seq_many_till -> B`:
  - **Parser:** Apply `A` zero more times, *until* the `B` accepts input
  - **Printer:** prints zero or more `A`, then `B`
- `A - lexeme -> B`:
  - Same as `seq`, but whitespace allowed
- `A - lex_many_till -> B`:
  - Same as `seq_many_till`, but whitespace allowed

DRAFT

The Welkin Grammar

```
start := units - lexeme -> EOF,
units := "" | {unit - lexeme -> *{"",
"," - lexeme -> units}},
unit := node - lexeme -> *{
  ":=" - lexeme -> choices,
  *{"", arc - lexeme -> chain}
},
choices := *{"", "|"} - lexeme ->
choices_list,
choices_list := *{"",
  chain - lexeme -> *{"", "|" - lexeme
-> choice_list}
},
chain := *{"", "*"} - seq -> path -
lexeme -> *{"", arc - lexeme ->
chain},
arc := right_arc | other_arc,
right_arc := "-" - seq -> node - seq -
> "->",
other_arc := "<-" - seq -> node - seq
-> *{"-", "->"},
node := *{"", "*"} - seq -> path,
path := *{"", modifiers} - seq ->
trailer - seq -> *{"", graph},
modifiers := *{
  "@",
  "~" - seq -> *{"", "@"}
},
trailer := segment - seq -> *{"", "."
- seq -> trailer},
segment := HANDLE - lexeme -> *{"",
graph},
graph :=  "{" - lexeme -> units -
lexeme -> "}",
HANDLE := ID | STRING,
STRING := SQ_STRING | DQ_STRING,
SQ_STRING := "'" - seq -> *{SQ_CHAR,
"\'"} - seq_many_till -> "'",
DQ_STRING := '"' - seq -> *{DQ_CHAR,
'\"'} - seq_many_till -> '"',
ID := ID_CHAR - seq_many_till ->
*{RESERVED, WHITESPACE}
```

Listing 1: Full Welkin grammar.

DRAFT

Outline of Unambiguity

- Simple grammar class: LL(1)

DRAFT

[1]Consult: Aho et al. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.).
[2]Edelmann et al. (2020). *Zippy LL(1) parsing with derivatives*. PLDI 2020.

Outline of Unambiguity

- Simple grammar class: LL(1)
- Desirable properties:[1]
  - Unambiguous: every valid input produces *one* output

DRAFT

[1]Consult: Aho et al. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.).
[2]Edelmann et al. (2020). *Zippy LL(1) parsing with derivatives*. PLDI 2020.

Outline of Unambiguity

- Simple grammar class: LL(1)
- Desirable properties:[1]
  - Unambiguous: every valid input produces *one* output
  - Efficient parsing
- For our parsers, need Edelmann et. al:[2]

DRAFT

[1] Consult: Aho et al. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.).
[2] Edelmann et al. (2020). *Zippy LL(1) parsing with derivatives*. PLDI 2020.

Outline of Unambiguity

- Simple grammar class: LL(1)
- Desirable properties:[1]
  - Unambiguous: every valid input produces *one* output
  - Efficient parsing
- For our parsers, need Edelmann et. al:[2]
  - Involves recursive equations on sets

DRAFT

[1] Consult: Aho et al. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.).
[2] Edelmann et al. (2020). *Zippy LL(1) parsing with derivatives*. PLDI 2020.

Outline of Unambiguity

- Simple grammar class: LL(1)
- Desirable properties:[1]
  - Unambiguous: every valid input produces *one* output
  - Efficient parsing
- For our parsers, need Edelmann et. al:[2]
  - Involves recursive equations on sets
  - Need: certain sets are djsoint

DRAFT

[1] Consult: Aho et al. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.).
[2] Edelmann et al. (2020). *Zippy LL(1) parsing with derivatives*. PLDI 2020.

Outline of Unambiguity

- Simple grammar class: LL(1)
- Desirable properties:[1]
  - Unambiguous: every valid input produces *one* output
  - Efficient parsing
- For our parsers, need Edelmann et. al:[2]
  - Involves recursive equations on sets
  - Need: certain sets are djsoint
  - For details, refer to thesis

DRAFT

[1] Consult: Aho et al. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.).
[2] Edelmann et al. (2020). *Zippy LL(1) parsing with derivatives*. PLDI 2020.

Agenda

Introduction

Language Overview

Syntax

Metatheory

Conclusion

DRAFT

Primer

- Terminology
  - **RE:** accepted by some Turing machine
    - Might not halt!
  - **Formal system:**
    - A decidable set of **formulas**

DRAFT

Primer

- Terminology
  - **RE:** accepted by some Turing machine
    - Might not halt!
  - **Formal system:**
    - A decidable set of **formulas**
    - A RE set of **rules**

DRAFT

Primer

- Terminology
  - **RE:** accepted by some Turing machine
    - Might not halt!
  - **Formal system:**
    - A decidable set of **formulas**
    - A RE set of **rules**
    - A computable **verifier** on **derivations**

DRAFT

Primer

- Terminology
  - **RE:** accepted by some Turing machine
    - Might not halt!
  - **Formal system:**
    - A decidable set of **formulas**
    - A RE set of **rules**
    - A computable **verifier** on **derivations**
  - **First Order Logic:**

DRAFT

Primer

- Terminology
  - **RE:** accepted by some Turing machine
    - Might not halt!
  - **Formal system:**
    - A decidable set of **formulas**
    - A RE set of **rules**
    - A computable **verifier** on **derivations**
  - **First Order Logic:**
    - Propositional logic (and, or, not)

DRAFT

Primer

- Terminology
  - **RE:** accepted by some Turing machine
    - Might not halt!
  - **Formal system:**
    - A decidable set of **formulas**
    - A RE set of **rules**
    - A computable **verifier** on **derivations**
  - **First Order Logic:**
    - Propositional logic (and, or, not)
    - Quantifiers (for all, exists)

DRAFT

Primer

- Terminology
  - **RE:** accepted by some Turing machine
    - Might not halt!
  - **Formal system:**
    - A decidable set of **formulas**
    - A RE set of **rules**
    - A computable **verifier** on **derivations**
  - **First Order Logic:**
    - Propositional logic (and, or, not)
    - Quantifiers (for all, exists)
  - **Peano Arithmetic** (PA): natural numbers + induction
  - **Consistency:** no proofs contain falsehood ($\perp$)

DRAFT

Formal System Limits

- Many theorems on limit:s

DRAFT

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Formal System Limits

- Many theorems on limit:s
  - **Gödel incompleteness**

DRAFT

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Formal System Limits

- Many theorems on limit:s
  - **Gödel incompleteness**
    - **First:** PA can't prove everything

DRAFT

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Formal System Limits

- Many theorems on limit:s
  - **Gödel incompleteness**
    - **First:** PA can't prove everything
    - **Second**: PA cannot prove *own* consistency

DRAFT

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Formal System Limits

- Many theorems on limit:s
  - **Gödel incompleteness**
    - **First:** PA can't prove everything
    - **Second**: PA cannot prove *own* consistency
  - Tarski undefinability

DRAFT

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Formal System Limits

- Many theorems on limit:s
  - **Gödel incompleteness**
    - **First:** PA can't prove everything
    - **Second**: PA cannot prove *own* consistency
  - Tarski undefinability
    - Cannot define all PA-truths *in* PA
    - Tarski's Criterion:

DRAFT

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Formal System Limits

- Many theorems on limit:s
  - **Gödel incompleteness**
    - **First:** PA can't prove everything
    - **Second**: PA cannot prove *own* consistency
  - Tarski undefinability
    - Cannot define all PA-truths *in* PA
    - Tarski's Criterion: $T(\varphi) \Leftrightarrow \varphi$

DRAFT

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Formal System Limits

- Many theorems on limit:s
  - **Gödel incompleteness**
    - **First:** PA can't prove everything
    - **Second**: PA cannot prove *own* consistency
  - Tarski undefinability
    - Cannot define all PA-truths *in* PA
    - Tarski's Criterion: $T(\varphi) \Leftrightarrow \varphi$
  - Etc.

DRAFT

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Formal System Limits

- Many theorems on limit:s
  - **Gödel incompleteness**
    - **First:** PA can't prove everything
    - **Second**: PA cannot prove *own* consistency
  - Tarski undefinability
    - Cannot define all PA-truths *in* PA
    - Tarski's Criterion: $T(\varphi) \Leftrightarrow \varphi$
  - Etc.
- Usual approach: *chain of stronger theories*

DRAFT

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Formal System Limits

- Many theorems on limit:s
  - **Gödel incompleteness**
    - **First:** PA can't prove everything
    - **Second**: PA cannot prove *own* consistency
  - Tarski undefinability
    - Cannot define all PA-truths *in* PA
    - Tarski's Criterion: $T(\varphi) \Leftrightarrow \varphi$
  - Etc.
- Usual approach: *chain of stronger theories*
  - Extend by $\mathrm{Con}(T)$ (encoding of consistency)

DRAFT

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Formal System Limits

- Many theorems on limit:s
  - **Gödel incompleteness**
    - **First:** PA can't prove everything
    - **Second**: PA cannot prove *own* consistency
  - Tarski undefinability
    - Cannot define all PA-truths *in* PA
    - Tarski's Criterion: $T(\varphi) \Leftrightarrow \varphi$
  - Etc.
- Usual approach: *chain of stronger theories*
  - Extend by $\mathrm{Con}(T)$ (encoding of consistency)
  - *Technically* enough, WITH non-constructive methods[1]

DRAFT

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Formal System Limits

- Many theorems on limit:s
  - **Gödel incompleteness**
    - **First:** PA can't prove everything
    - **Second**: PA cannot prove *own* consistency
  - Tarski undefinability
    - Cannot define all PA-truths *in* PA
    - Tarski's Criterion: $T(\varphi) \Leftrightarrow \varphi$
  - Etc.
- Usual approach: *chain of stronger theories*
  - Extend by $\mathrm{Con}(T)$ (encoding of consistency)
  - *Technically* enough, WITH non-constructive methods[1]
  - Problem: want to identify + trust proofs!

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

DRAFT

Artëmov's Selector Proofs

DRAFT

Artëmov's Selector Proofs

- Artëmov *changes* this paradigm

DRAFT

Artëmov's Selector Proofs

- Artëmov *changes* this paradigm
  - There is a *weaker* proof: **selectors**

DRAFT

Artëmov's Selector Proofs

- Artëmov *changes* this paradigm
  - There is a *weaker* proof: **selectors**
- **Selector:** algorithm that takes proofs as inputs

DRAFT

Artëmov's Selector Proofs

- Artëmov *changes* this paradigm
  - There is a *weaker* proof: **selectors**
- **Selector:** algorithm that takes proofs as inputs
  1. Takes in *any* proof

DRAFT

Artëmov's Selector Proofs

- Artëmov *changes* this paradigm
  - There is a *weaker* proof: **selectors**
- **Selector:** algorithm that takes proofs as inputs
  1. Takes in *any* proof
  2. If it accepts a proof, no $\perp$ appears

DRAFT

Artëmov's Selector Proofs

- Artëmov *changes* this paradigm
  - There is a *weaker* proof: **selectors**
- **Selector:** algorithm that takes proofs as inputs
  1. Takes in *any* proof
  2. If it accepts a proof, no $\perp$ appears
- Artëmov

DRAFT

Serial-Soundness

DRAFT

[1]Consult: Hájek & Pudlák (2017). Metamathematics of First-Order Arithmetic. Cambridge.

Serial-Soundness

- Extend Artëmov to **serial-soundness:**
  - Secret: *same* technique!
  - Use **partial truh predicates**[1]
    - Approximates truth up to n derviations
- **Serial soundness:** a first-order RE theory T with a selector s.t.:
  - Accepts all proofs + preserves truth

DRAFT

[1]Consult: Hájek & Pudlák (2017). Metamathematics of First-Order Arithmetic. Cambridge.

Serial-Soundness

- Extend Artëmov to **serial-soundness:**
  - Secret: *same* technique!
  - Use **partial truh predicates**[1]
    - Approximates truth up to n derviations
- **Serial soundness:** a first-order RE theory T with a selector s.t.:
  - Accepts all proofs + preserves truth
  - *cannot* be proven with:
    - Excluded Middle: $\varphi \vee \neg\varphi$

DRAFT

[1]Consult: Hájek & Pudlák (2017). Metamathematics of First-Order Arithmetic. Cambridge.

Serial-Soundness

- Extend Artëmov to **serial-soundness:**
  - Secret: *same* technique!
  - Use **partial truh predicates**[1]
    - Approximates truth up to n derviations
- **Serial soundness:** a first-order RE theory T with a selector s.t.:
  - Accepts all proofs + preserves truth
  - *cannot* be proven with:
    - Excluded Middle: $\varphi \vee \neg\varphi$
    - Explosion: $\bot \Rightarrow \varphi$

DRAFT

[1]Consult: Hájek & Pudlák (2017). Metamathematics of First-Order Arithmetic. Cambridge.

Serial-Soundness

- Extend Artëmov to **serial-soundness:**
  - Secret: *same* technique!
  - Use **partial truh predicates**[1]
    - Approximates truth up to n derviations
- **Serial soundness:** a first-order RE theory T with a selector s.t.:
  - Accepts all proofs + preserves truth
  - *cannot* be proven with:
    - Excluded Middle: $\varphi \vee \neg\varphi$
    - Explosion: $\bot \Rightarrow \varphi$
- **Self-verifying**: *in* PA, T can prove its own serial-soundness
  - PA $\vdash_T \varphi$ if $T \vdash \varphi$ and T self-verifying

DRAFT

[1]Consult: Hájek & Pudlák (2017). Metamathematics of First-Order Arithmetic. Cambridge.

Reliable Proofs

- How to generalize to *any* theory?

DRAFT

Reliable Proofs

- How to generalize to *any* theory?
  - Example: Naive Set Theory

DRAFT

Reliable Proofs

- How to generalize to *any* theory?
  - Example: Naive Set Theory
    - Can't *fully* trust b/c Russell's Paradox

DRAFT

Reliable Proofs

- How to generalize to *any* theory?
  - Example: Naive Set Theory
    - Can't *fully* trust b/c Russell's Paradox
    - But can trust, e.g., $x \in X \Leftrightarrow x \in X$

DRAFT

Reliable Proofs

- How to generalize to *any* theory?
  - Example: Naive Set Theory
    - Can't *fully* trust b/c Russell's Paradox
    - But can trust, e.g., $x \in X \Leftrightarrow x \in X$
    - Idea: check for sound *restricted* theory
- Let T_1 be an RE theory, φ a sentence

DRAFT

Reliable Proofs

- How to generalize to *any* theory?
  - Example: Naive Set Theory
    - Can't *fully* trust b/c Russell's Paradox
    - But can trust, e.g., $x \in X \Leftrightarrow x \in X$
    - Idea: check for sound *restricted* theory
- Let T_1 be an RE theory, φ a sentence
- **Reliable proof**: a triple (S, p_S, p_φ) s.t:
  - $S \supseteq T_1$, $S \neq \varnothing$

DRAFT

Reliable Proofs

- How to generalize to *any* theory?
  - Example: Naive Set Theory
    - Can't *fully* trust b/c Russell's Paradox
    - But can trust, e.g., $x \in X \Leftrightarrow x \in X$
    - Idea: check for sound *restricted* theory
- Let T_1 be an RE theory, φ a sentence
- **Reliable proof**: a triple (S, p_S, p_φ) s.t:
  - $S \supseteq T_1$, $S \neq \varnothing$
  - p_S: constructs some self-verifying $S' \supseteq S$

DRAFT

Reliable Proofs

- How to generalize to *any* theory?
  - Example: Naive Set Theory
    - Can't *fully* trust b/c Russell's Paradox
    - But can trust, e.g., $x \in X \Leftrightarrow x \in X$
    - Idea: check for sound *restricted* theory
- Let T_1 be an RE theory, φ a sentence
- **Reliable proof**: a triple (S, p_S, p_φ) s.t:
  - $S \supseteq T_1$, $S \neq \varnothing$
  - p_S: constructs some self-verifying $S' \supseteq S$
  - p_φ: S-proof of φ

DRAFT

Proof Completeness

- **Note:** still has incompleteness!

DRAFT

Proof Completeness

- **Note:** still has incompleteness!
  - Proofs are limited! (Not surprising)

DRAFT

Proof Completeness

- **Note:** still has incompleteness!
  - Proofs are limited! (Not surprising)
  - Also: not a *single* theory suffices

DRAFT

Proof Completeness

- **Note:** still has incompleteness!
  - Proofs are limited! (Not surprising)
  - Also: not a *single* theory suffices
  - But, a theories *can* be self-verifying!

DRAFT

Proof Completeness

- **Note:** still has incompleteness!
  - Proofs are limited! (Not surprising)
  - Also: not a *single* theory suffices
  - But, a theories *can* be self-verifying!
- How to prove?

DRAFT

Proof Completeness

- **Note:** still has incompleteness!
  - Proofs are limited! (Not surprising)
  - Also: not a *single* theory suffices
  - But, a theories *can* be self-verifying!
- How to prove?
  - Basic idea: ZFC is self-verifying!

DRAFT

Proof Completeness

- **Note:** still has incompleteness!
  - Proofs are limited! (Not surprising)
  - Also: not a *single* theory suffices
  - But, a theories *can* be self-verifying!
- How to prove?
  - Basic idea: ZFC is self-verifying!
    - Again, same proof from Artëmov!

DRAFT

Proof Completeness

- **Note:** still has incompleteness!
  - Proofs are limited! (Not surprising)
  - Also: not a *single* theory suffices
  - But, a theories *can* be self-verifying!
- How to prove?
  - Basic idea: ZFC is self-verifying!
    - Again, same proof from Artëmov!
  - HUGE leap; how to extend it?

DRAFT

Proof Completeness

- **Note:** still has incompleteness!
  - Proofs are limited! (Not surprising)
  - Also: not a *single* theory suffices
  - But, a theories *can* be self-verifying!
- How to prove?
  - Basic idea: ZFC is self-verifying!
    - Again, same proof from Artëmov!
  - HUGE leap; how to extend it?
  - Answer: comprehension!

DRAFT

Crash Course: Recursive Ordinals

DRAFT

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Crash Course: Recursive Ordinals

- Roughly: certain transitive sets ($x \in y \in z \Rightarrow x \in z$)

DRAFT

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Crash Course: Recursive Ordinals

- Roughly: certain transitive sets ($x \in y \in z \Rightarrow x \in z$)
  - Contents: enumerated by some Turing machine

DRAFT

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Crash Course: Recursive Ordinals

- Roughly: certain transitive sets ($x \in y \in z \Rightarrow x \in z$)
  - Contents: enumerated by some Turing machine
  - Generalized induction: **transfinite induction**
    - Adds a **limit case**
- Limit: **Church-Kleene Ordinal** (ω_1^{CK})

DRAFT

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Crash Course: Recursive Ordinals

- Roughly: certain transitive sets ($x \in y \in z \Rightarrow x \in z$)
  - Contents: enumerated by some Turing machine
  - Generalized induction: **transfinite induction**
    - Adds a **limit case**
- Limit: **Church-Kleene Ordinal** (ω_1^{CK})
- **Proof-theoretic ordinal:** measures theory strength
  - Important: this is recursive if T is RE *and* sound

DRAFT

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Crash Course: Recursive Ordinals

- Roughly: certain transitive sets ($x \in y \in z \Rightarrow x \in z$)
  - Contents: enumerated by some Turing machine
  - Generalized induction: **transfinite induction**
    - Adds a **limit case**
- Limit: **Church-Kleene Ordinal** (ω_1^{CK})
- **Proof-theoretic ordinal:** measures theory strength
  - Important: this is recursive if T is RE *and* sound
  - Bounded *strictly* below ω_1^{CK}!
  - Using inducitve step: can reach Feferman-Schütte ordinal Γ_0[1]
  - For us: how to reach *anywhere* below ω_1^{CK}?

DRAFT

[1] Feferman (1962). Transfinite Recursive Progressions of Axiomatic Theories. J. Symb. Log. 27(3).

Crash Course: Simpson Comprehension

- Enter Simpson[1]

DRAFT

[1]Simpson (2009). Subsystems of Second Order Arithmetic (2nd ed.). Cambridge
[2]Kleene (1944). On the Forms of the Predicates in the Theory of Constructive Ordinals. Am. J. Math. 66(1).

Crash Course: Simpson Comprehension

- Enter Simpson[1]
- Simpson worked on *Reverse Mathematics*
  - Studies the axioms necessary to prove theorems
- We need his comprehension axiom:

$$\exists X.\forall n.(n \in X \Leftrightarrow \Phi(n))$$

  - Φ: very complex formula (Δ^1_1)
    - Also called **hyperarithmetic**
    - By Kleene, this reaches everywhere below ω_1^{CK}![2]

[1]Simpson (2009). Subsystems of Second Order Arithmetic (2nd ed.). Cambridge
[2]Kleene (1944). On the Forms of the Predicates in the Theory of Constructive Ordinals. Am. J. Math. 66(1).

DRAFT

Proof Completeness

DRAFT

[1]Proven in: Kleene (1944). On the Forms of the Predicates in the Theory of Constructive Ordinals. Am. J. Math. 66(1).

Proof Completeness

- Time for construction!:

DRAFT

[1] Proven in: Kleene (1944). On the Forms of the Predicates in the Theory of Constructive Ordinals. Am. J. Math. 66(1).

Proof Completeness

- Time for construction!:
- Let:

DRAFT

[1] Proven in: Kleene (1944). On the Forms of the Predicates in the Theory of Constructive Ordinals. Am. J. Math. 66(1).

Proof Completeness

- Time for construction!:
- Let:
  - λ be limit of recursive ordinals $\beta_1, \beta_2, \ldots$

DRAFT

[1] Proven in: Kleene (1944). On the Forms of the Predicates in the Theory of Constructive Ordinals. Am. J. Math. 66(1).

Proof Completeness

- Time for construction!:
- Let:
  - λ be limit of recursive ordinals $\beta_1, \beta_2, \ldots$
  - Each T_{β_i} self-verifying

DRAFT

[1] Proven in: Kleene (1944). On the Forms of the Predicates in the Theory of Constructive Ordinals. Am. J. Math. 66(1).

Proof Completeness

- Time for construction!:
- Let:
  - λ be limit of recursive ordinals $\beta_1, \beta_2, \ldots$
  - Each T_{β_i} self-verifying
  - $T' = \bigcup_{\beta<\lambda} T_\beta$

DRAFT

[1] Proven in: Kleene (1944). On the Forms of the Predicates in the Theory of Constructive Ordinals. Am. J. Math. 66(1).

Proof Completeness

- Time for construction!:
- Let:
  - λ be limit of recursive ordinals $\beta_1, \beta_2, \ldots$
  - Each T_{β_i} self-verifying
  - $T' = \bigcup_{\beta<\lambda} T_\beta$
  - $\mathrm{Comp}_\lambda(\Delta^1_1) = \exists X_{\varphi,\lambda}.\forall n.(n \in X_{\varphi,\lambda} \Leftrightarrow \mathrm{PA} \vdash_{T'} \varphi(n))$

DRAFT

[1] Proven in: Kleene (1944). On the Forms of the Predicates in the Theory of Constructive Ordinals. Am. J. Math. 66(1).

Proof Completeness

- Time for construction!:
- Let:
  - λ be limit of recursive ordinals $\beta_1, \beta_2, \ldots$
  - Each T_{β_i} self-verifying
  - $T' = \bigcup_{\beta<\lambda} T_\beta$
  - $\mathrm{Comp}_\lambda(\Delta^1_1) = \exists X_{\varphi,\lambda}.\forall n.(n \in X_{\varphi,\lambda} \Leftrightarrow \mathrm{PA} \vdash_{T'} \varphi(n)))$
- Our construction: T_λ

DRAFT

[1] Proven in: Kleene (1944). On the Forms of the Predicates in the Theory of Constructive Ordinals. Am. J. Math. 66(1).

Proof Completeness

- Time for construction!:
- Let:
  - λ be limit of recursive ordinals $\beta_1, \beta_2, \ldots$
  - Each T_{β_i} self-verifying
  - $T' = \bigcup_{\beta<\lambda} T_\beta$
  - $\mathrm{Comp}_\lambda(\Delta^1_1) = \exists X_{\varphi,\lambda}.\forall n.(n \in X_{\varphi,\lambda} \Leftrightarrow \mathrm{PA} \vdash_{T'} \varphi(n))$
- Our construction: T_λ
  - ✓ Eventually reaches limit[1]

[1] Proven in: Kleene (1944). On the Forms of the Predicates in the Theory of Constructive Ordinals. Am. J. Math. 66(1).

DRAFT

Proof Completeness

- Time for construction!:
- Let:
  - λ be limit of recursive ordinals $\beta_1, \beta_2, \ldots$
  - Each T_{β_i} self-verifying
  - $T' = \bigcup_{\beta<\lambda} T_\beta$
  - $\mathrm{Comp}_\lambda(\Delta^1_1) = \exists X_{\varphi,\lambda}.\forall n.(n \in X_{\varphi,\lambda} \Leftrightarrow \mathrm{PA} \vdash_{T'} \varphi(n))$
- Our construction: T_λ
  - ✓ Eventually reaches limit[1]
  - ✓ Self-verifying

DRAFT

[1] Proven in: Kleene (1944). On the Forms of the Predicates in the Theory of Constructive Ordinals. Am. J. Math. 66(1).

Proof Completeness

- Time for construction!:
- Let:
  - λ be limit of recursive ordinals $\beta_1, \beta_2, \ldots$
  - Each T_{β_i} self-verifying
  - $T' = \bigcup_{\beta<\lambda} T_\beta$
  - $\mathrm{Comp}_\lambda(\Delta^1_1) = \exists X_{\varphi,\lambda}.\forall n.(n \in X_{\varphi,\lambda} \Leftrightarrow \mathrm{PA} \vdash_{T'} \varphi(n))$
- Our construction: T_λ
  - ✓ Eventually reaches limit[1]
  - ✓ Self-verifying
  - ✓ **Constructive**

[1] Proven in: Kleene (1944). On the Forms of the Predicates in the Theory of Constructive Ordinals. Am. J. Math. 66(1).

DRAFT

Proof Completeness

- Time for construction!:
- Let:
  - λ be limit of recursive ordinals $\beta_1, \beta_2, \ldots$
  - Each T_{β_i} self-verifying
  - $T' = \bigcup_{\beta<\lambda} T_\beta$
  - $\mathrm{Comp}_\lambda(\Delta^1_1) = \exists X_{\varphi,\lambda}.\forall n.(n \in X_{\varphi,\lambda} \Leftrightarrow \mathrm{PA} \vdash_{T'} \varphi(n)))$
- Our construction: T_λ
  - ✓ Eventually reaches limit[1]
  - ✓ Self-verifying
  - ✓ **Constructive**
- TL;DR: Artëmov + Simpson + Reliability = Proofs 💯

[1] Proven in: Kleene (1944). On the Forms of the Predicates in the Theory of Constructive Ordinals. Am. J. Math. 66(1).

DRAFT

Agenda

Introduction

Language Overview

Syntax

Metatheory

Conclusion

DRAFT

Comparisons to Existing Solutions

DRAFT

Comparisons to Existing Solutions

- **Resource Description Framework:**

DRAFT

Comparisons to Existing Solutions

- **Resource Description Framework:**
  - Similar ideas/terms about representations

DRAFT

Comparisons to Existing Solutions

- **Resource Description Framework:**
  - Similar ideas/terms about representations
  - Welkin:

DRAFT

Comparisons to Existing Solutions

- **Resource Description Framework:**
  - Similar ideas/terms about representations
  - Welkin:
    - Higher order terms

DRAFT

Comparisons to Existing Solutions

- **Resource Description Framework:**
  - Similar ideas/terms about representations
  - Welkin:
    - Higher order terms
    - Better ogical reasoning

DRAFT

Comparisons to Existing Solutions

- **Resource Description Framework:**
  - Similar ideas/terms about representations
  - Welkin:
    - Higher order terms
    - Better ogical reasoning
- **Labeled Property Graphs:**

DRAFT

Comparisons to Existing Solutions

- **Resource Description Framework:**
  - Similar ideas/terms about representations
  - Welkin:
    - Higher order terms
    - Better ogical reasoning
- **Labeled Property Graphs:**
  - Simple core idea + expressive

DRAFT

Comparisons to Existing Solutions

- **Resource Description Framework:**
  - Similar ideas/terms about representations
  - Welkin:
    - Higher order terms
    - Better ogical reasoning
- **Labeled Property Graphs:**
  - Simple core idea + expressive
  - Welkin:

DRAFT

Comparisons to Existing Solutions

- **Resource Description Framework:**
  - Similar ideas/terms about representations
  - Welkin:
    - Higher order terms
    - Better ogical reasoning
- **Labeled Property Graphs:**
  - Simple core idea + expressive
  - Welkin:
    - Formal definitions and grammar

DRAFT

Comparisons to Existing Solutions

- **Resource Description Framework:**
  - Similar ideas/terms about representations
  - Welkin:
    - Higher order terms
    - Better ogical reasoning
- **Labeled Property Graphs:**
  - Simple core idea + expressive
  - Welkin:
    - Formal definitions and grammar
    - Logical engine

DRAFT

Comparisons to Existing Solutions (cont')

DRAFT

Comparisons to Existing Solutions (cont')

- **Cyc:**

DRAFT

Comparisons to Existing Solutions (cont')

- **Cyc:**
  - Similarly depend on contexts

DRAFT

Comparisons to Existing Solutions (cont')

- **Cyc:**
  - Similarly depend on contexts
  - Welkin:

DRAFT

Comparisons to Existing Solutions (cont')

- **Cyc:**
  - Similarly depend on contexts
  - Welkin:
    - Flexible data format

DRAFT

Comparisons to Existing Solutions (cont')

- **Cyc:**
  - Similarly depend on contexts
  - Welkin:
    - Flexible data format
    - Open specification

DRAFT

Future Work

DRAFT

Future Work

- Write the Standard

DRAFT

Future Work

- Write the Standard
- Create a Fast Implementation

DRAFT

Future Work

- Write the Standard
- Create a Fast Implementation
- Query Solvers

DRAFT

Future Work

- Write the Standard
- Create a Fast Implementation
- Query Solvers
- Revision System

DRAFT

Future Work

- Write the Standard
- Create a Fast Implementation
- Query Solvers
- Revision System
- Package Management

DRAFT

Future Work

- Write the Standard
- Create a Fast Implementation
- Query Solvers
- Revision System
- Package Management

DRAFT

Thank you!

Questions?

DRAFT